Claude Code
핵심 가이드

설치부터 Agent Team까지, AI 코딩 도구의 모든 것

2026년 2월 I Opus 4.6 기준

프로젝트 브레인 • Plan Mode • Skill • MCP • Sub Agent • Agent Team

목차

Part 1. Claude Code 시작하기

› Claude Code란?
› 구독 플랜 및 설치
› 개발 환경 (IDE) 설정
› 토큰과 컨텍스트 윈도우
› 프로젝트 브레인 (CLAUDE.md)
› CLAUDE.md 작성 실전 팁
› CLAUDE.md 레이어 구조
› Rules 폴더와 Auto Memory

Part 2. 실전 개발 워크플로우

› Task-Do-Verify 루프
› Context Rot 방지하기
› 병렬 작업과 Hook 시스템
› 4가지 권한 모드
› Plan Mode 활용법
› 컨텍스트 관리 전략
› 주요 슬래시 명령어

Part 3. 자동화와 확장

› Skill 시스템
› MCP (Model Context Protocol)
› Sub Agent (서브 에이전트)
› Agent Team
› Git Worktree
› 전체 워크플로우 조합

부록

› 주요 키보드 단축키
› 액션 아이템 3가지
› 보안 주의사항

PART 1

Claude Code 시작하기

설치, 개발 환경 설정, 그리고 AI의 성능을 결정짓는 프로젝트 브레인까지

Claude Code란?

Claude Code는 Anthropic이 만든 **에이전틱(Agentic) 코딩 도구**입니다. 터미널에서 동작하며 코드베이스를 읽고, 파일을 편집하고, 명령어를 실행하고, 개발 도구와 통합됩니다. 자연어로 기능 구현, 버그 수정, 테스트 작성을 요청할 수 있고, Git 커밋/브랜치 생성/PR 오픈까지 자동화할 수 있습니다.

터미널, IDE(VS Code, JetBrains), 데스크톱 앱, 웹 브라우저, 모바일(iOS) 등 다양한 환경에서 사용 가능합니다.

구독 플랜 및 설치

플랜	가격	주요 특징
Pro	$20/월	모든 기능 사용 가능, 사용량 제한 있음
Max 5x	$100/월	Pro의 5배 사용량, Opus 4.6 접근, Agent Teams
Max 20x	$200/월	Pro의 20배 사용량, 최대 우선순위
API	종량제	Sonnet 4.5: 입력 $3/1M, 출력 $15/1M 토큰

설치 명령어

```
# macOS / Linux / WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# Homebrew
brew install --cask claude-code
```

> **Tip:** 네이티브 설치는 Node.js가 필요 없으며, 자동 업데이트가 내장되어 있습니다. 설치 후 프로젝트 폴더에서 `claude` 명령어로 바로 시작할 수 있습니다.

개발 환경 (IDE) 설정

Claude Code는 터미널에서 직접 쓸 수도 있지만, **VS Code**에서 사용하는 것이 가장 편합니다. VS Code에 Anthropic 공식 Claude Code 확장 프로그램을 설치하면 됩니다.

> **주의:** 반드시 **Anthropic 공식** 확장 프로그램을 설치하세요. 비슷한 이름의 비공식 확장에는 악성 코드가 포함되어 있을 수 있습니다.

IDE 화면 구성

영역	위치	역할
파일 탐색기	왼쪽	프로젝트의 모든 파일/폴더 탐색
코드 편집기	가운데	파일 내용 확인 및 직접 수정
Claude Code 채팅	오른쪽/아래	AI에게 명령 전달, 결과 확인

Claude Code는 현재 열려 있는 파일을 자동으로 인식합니다. 특정 파일에 대해 작업을 시키려면 해당 파일을 편집기에서 열어둔 상태로 대화하세요.

토큰과 컨텍스트 윈도우

토큰 (Token)

AI가 텍스트를 처리하는 기본 단위. 대략 한 단어가 한 토큰 정도입니다. 자동차에 비유하면 **연료**에 해당합니다.

컨텍스트 윈도우

AI가 한 번에 기억할 수 있는 대화의 양. Opus 4.6 기준 **200,000 토큰**(약 150,000 단어). 자동차의 **연료 탱크**에 해당합니다.

> **핵심:** 이 탱크를 얼마나 효율적으로 쓰느냐가 Claude Code 활용의 핵심입니다.

프로젝트 브레인 (CLAUDE.md)

`CLAUDE.md`는 프로젝트 폴더 최상위에 위치하는 파일로, Claude Code가 대화를 시작할 때 **가장 먼저 읽는 작업 지침서**입니다. 기술 스택, 코드 스타일, 프로젝트 구조 등의 정보를 담습니다.

> **비유:** CLAUDE.md는 **배의 방향타**입니다. 출발할 때 각도를 1도만 틀어도 수천 킬로미터를 항해하면 도착지가 완전히 달라집니다. AI와 대화를 시작하기 전에 이 파일이 먼저 주입되므로, 여기에 뭘 적느냐에 따라 전체 프로젝트의 방향이 결정됩니다.

CLAUDE.md 생성 방법

1. `/init` 명령어로 자동 생성 (프로젝트 구조, 기술 스택, 아키텍처 자동 분석)
2. 직접 규칙 추가 (테스트 코드 작성, 함수 이름 규칙, 에러 처리 패턴 등)
3. **200~500줄** 사이로 유지 (너무 길면 AI가 중간 내용을 놓침)

CLAUDE.md 작성 실전 팁

#	팁	이유
1	가장 중요한 규칙은 **파일 맨 위에** 배치	AI 모델의 Primacy Bias - 시작/끝 부분을 가장 잘 기억
2	API 문서 전체를 복사하지 말 것	토큰 낭비. 핵심 정보만 간결하게
3	같은 실수 2~3번 반복 시 규칙 추가	실제 문제가 생길 때 하나씩 추가하는 게 효과적
4	주기적으로 불필요한 규칙 정리	CLAUDE.md도 기술 부채가 쌓일 수 있음

CLAUDE.md 3가지 레이어

레이어	위치	적용 범위
글로벌	`~/.claude/CLAUDE.md`	모든 프로젝트 공통 (개인 스타일, 코딩 규칙)
프로젝트	프로젝트 루트 `CLAUDE.md`	해당 프로젝트만 적용
관리자	기업용 전용	조직 전체 정책 강제

세 레이어가 병합되어 최종 지침이 완성됩니다. 프로젝트별로 다른 설정이 필요하면 프로젝트 레이어에서 덮어씁니다.

Rules 폴더와 Auto Memory

Rules 폴더

`.claude/rules/` 폴더에 규칙을 주제별로 분리 저장할 수 있습니다. 설정 파일을 모듈화하는 것과 같은 개념입니다.

```
.claude/
  rules/
    workflow.md           # 워크플로우 규칙
    design-rules.md       # 디자인 규칙
    technical-defaults.md # 기술 기본 설정
```

Auto Memory

`MEMORY.md` 파일에 중요한 정보를 자동 저장하는 기능입니다. 세션이 끝나도 다음 대화에서 이 정보를 기억합니다.

> **Part 1 핵심 요약:** Claude Code의 성능은 코드를 입력하기 전에 이미 결정됩니다. CLAUDE.md, Rules 폴더, 메모리를 제대로 설정하면 AI가 프로젝트를 정확히 이해한 상태에서 출발합니다. **설정에 5분 투자하면 이후 수십 시간을 아낄 수 있습니다.**

PART 2

실전 개발 워크플로우

검증 루프, Plan Mode, 컨텍스트 관리로 AI 코딩의 품질과 속도를 모두 잡는 법

Task-Do-Verify 루프

AI 코딩의 핵심 워크플로우입니다. 대부분의 사람들이 AI 코딩에 실망하는 이유는 바로 **검증 단계를 빼먹기 때문**입니다.

Task (작업 지시) → Do (AI가 실행) → Verify (결과 검증) → 반복

디자인 작업

1. 원하는 디자인 스크린샷 전달
2. 결과물을 다시 스크린샷으로 찍기
3. 원본과 비교하여 수정 지시
4. 서너 번 반복으로 완성

코딩 작업

1. 테스트 코드 자동 작성/실행 지시
2. 테스트 통과 시 다음 단계
3. 실패 시 AI가 자동 수정
4. TDD(테스트 주도 개발) 방식

AI의 진짜 가치: 완벽함이 아닌 **속도**에 있습니다. 80% 완성도에서 시작해서 빠르게 수렴하는 것이 핵심입니다.

Context Rot 방지하기

AI 코딩에서 가장 흔한 실패 패턴: 처음에 잘 되다가 30분 후 갑자기 아무것도 동작하지 않는 현상.

Context Rot이란? 오류가 누적되면서 AI가 자기가 이전에 뭘 했는지 혼란스러워하는 현상입니다. 매 단계마다 검증하는 습관이 이 문제를 근본적으로 방지합니다. **귀찮아도 매번 검증하는 습관이 결국 가장 빠른 길입니다.**

병렬 작업과 Hook 시스템

병렬 작업

IDE에서 탭을 여러 개 열어 각각 다른 작업을 시킬 수 있습니다. 추천 동시 실행 수는 **3~4개**입니다.

Hook 시스템

Claude Code가 특정 동작을 할 때 자동으로 실행되는 사용자 정의 스크립트입니다.

Hook 이벤트	발생 시점	활용 예시
`PreToolUse`	도구 실행 전	보호 파일 편집 차단

`PostToolUse`	도구 실행 후	자동 코드 포맷팅
`Stop`	응답 완료 시	완료 알림 소리
`Notification`	알림 전송 시	macOS 알림
`SessionStart`	세션 시작 시	환경 초기화
`UserPromptSubmit`	프롬프트 제출 시	입력 검증

Hook 종료 코드

코드	동작
Exit 0	작업 진행 허용
Exit 2	작업 차단 (stderr 메시지가 Claude에 피드백)
기타	작업 진행, stderr는 로그에만 기록

4가지 권한 모드

모드	설명	추천 대상
기본 모드	파일 변경마다 허락 구함	안전 최우선
자동 수정 모드	기존 파일 수정 자동, 새 파일만 확인	초보자 추천
Bypass Permissions	모든 권한 AI에게 위임	격리 환경만
Plan Mode	설계를 먼저, 코드 수정 불가	복잡한 작업

> **Bypass Permissions 주의:** AI가 시스템 파일을 통째로 날려버린 사례가 보고된 적 있습니다. 격리된 환경에서만 사용하세요.

Plan Mode - 가장 강력한 기능

짓기 전에 설계도를 먼저 그리는 방식. Plan Mode에서 Claude Code는 코드를 직접 수정하지 않습니다.

1. 프로젝트 구조를 읽고 분석합니다
2. 질문이 있으면 사용자에게 물어봅니다
3. 어떻게 구현할지 상세한 계획을 세웁니다
4. 사용자가 검토하고 승인하면 코드를 작성합니다

> **효과:** 35분 걸리는 작업이 15분으로 단축. 계획 1분 투자 = 빌드 10분 절약. 특히 데이터베이스, 결제, 인증 같은 복잡한 기능에서 효과 극대화.

> **비유:** 플랜 없이 코딩하는 건 네비게이션 없이 처음 가는 곳을 운전하는 것과 같습니다. 도착은 하겠지만, 기름값이 3배로 나옵니다.

컨텍스트 관리 전략

첫 메시지를 입력하기도 전에 이미 **25% 이상**이 소비되어 있습니다 (시스템 프롬프트, 도구 설명, MCP, 메모리 등).

#	전략	설명
1	`/context`	현재 컨텍스트 사용량 확인
2	`/compact`	대화를 고밀도 요약으로 압축
3	짧고 구체적인 프롬프트	핵심만 전달하여 토큰 절약
4	Sub Agent 활용	리서치를 별도 에이전트에 위임, 메인 컨텍스트 보호
5	Extended Thinking	사고 과정 토큰이 컨텍스트에 누적되지 않음
6	모델 적재적소	단순 작업 Sonnet, 복잡한 작업 Opus

주요 슬래시 명령어

명령어	설명
`/init`	CLAUDE.md 자동 생성
`/context`	컨텍스트 사용량 확인
`/compact`	대화 압축
`/clear`	대화 완전 초기화
`/cost`	토큰 비용 확인
`/model`	모델 변경 (Sonnet / Opus / Haiku)
`/review`	최근 변경사항 코드 리뷰
`/config`	설정 인터페이스
`/permissions`	권한 규칙 관리
`/hooks`	Hook 설정
`/agents`	서브에이전트 관리
`/add-dir`	추가 디렉토리 포함

PART 3

자동화와 확장

Skill, MCP, Sub Agent, Agent Team으로 개인이 팀 수준의 생산성을 내는 법

Skill 시스템

Skill은 **AI를 위한 업무 매뉴얼**입니다. 반복 작업을 단계별 체크리스트로 파일에 저장합니다. `/스킬이름`으로 바로 실행할 수 있습니다.

Skill 구조

```
# 저장 위치
.claude/skills/my-skill.skill.md

# 파일 형식 (Markdown)
---
name: my-skill
description: 이 스킬의 간단한 설명
---
실제 작업 단계가 여기에 적힙니다...
```

> **영리한 설계:** Front Matter(이름/설명)만 먼저 로드하므로 약 60~70 토큰만 소비. 실제 내용은 호출 시에만 로드. **Skill을 100개 만들어도 컨텍스트를 거의 차지하지 않습니다.**

Skill 활용 사례

Skill	수동 작업	Skill 적용
리드 스크래핑	30분+	90초
이메일 분류/답장	1시간+	수 분
주제별 리서치	수 시간	수 분
맞춤 웹사이트 생성	수 일	30초

Skill 만드는 방법

1. 작업을 음성이나 텍스트로 Claude에게 설명
2. Claude가 Skill 파일 형식으로 자동 정리
3. 70% 정확도에서 시작 (괜찮습니다!)
4. 테스트하며 실패 부분 수정 (2~3회 반복)
5. 98~99% 정확도 달성

> **Tip:** 새로 만든 Skill은 반드시 **새로운 세션**에서 테스트하세요. 기존 대화 컨텍스트가 결과에 영향을 줄 수 있습니다.

MCP (Model Context Protocol)

MCP는 **다른 사람이 만들어 놓은 Skill**과 비슷합니다. Gmail, Slack, Notion, Google Sheets 등 외부 서비스와 Claude Code를 연결하는 플러그인입니다.

MCP vs Skill 비교

항목	MCP	Skill
토큰 사용	수천 토큰	60 토큰
처리 속도	이메일당 수 초	0.3초
용도	**프로토타입용**	**운영용**

> **추천 워크플로우:** MCP로 빠르게 기능 테스트 → 잘 작동하면 API 직접 호출하는 Skill로 변환 → 토큰 사용량 극적 감소

> **최추천 MCP:** Anthropic Chrome DevTools MCP. Chrome 브라우저를 직접 제어하여 웹 페이지 열기, 버튼 클릭, 데이터 추출이 가능합니다. API가 없는 서비스에서 데이터를 가져올 때 필수입니다.

Sub Agent (서브 에이전트)

메인 Claude Code가 특정 업무를 **별도 에이전트에게 위임**하는 개념입니다. Sub Agent는 자기만의 별도 컨텍스트에서 작업하고, 결과 요약만 메인에게 돌려줍니다.

내장 Sub Agent

에이전트	모델	용도
Explore	Haiku (빠름/저렴)	코드베이스 탐색, 파일 검색
Plan	상속	Plan Mode에서 리서치
General-purpose	상속	복잡한 멀티 스텝 작업
Bash	상속	별도 컨텍스트에서 명령어 실행

유용한 커스텀 Sub Agent

에이전트	역할
리서치 에이전트	저렴한 Sonnet으로 방대한 자료 검색/요약
코드 리뷰 에이전트	새로운 시각에서 코드 품질 검토
품질 보증 에이전트	테스트 코드 자동 작성/실행

> **확률의 수학:** 각 Sub Agent 성공률 95%일 때, 10개를 동시에 돌리면 전체 성공률은 **59%**입니다 (0.95의 10제곱). 작업을 잘게 쪼개서 단

> 순한 일을 시키는 것이 핵심입니다.

Agent Team

Sub Agent의 진화형. 팀원들이 **서로 소통**하며 자율적으로 협업합니다.

Agent Team 특징

특징	설명
독립 인스턴스	각 팀원이 자기만의 컨텍스트, CLAUDE.md, MCP 접근 권한 보유
자율 협업	공유 태스크 보드에서 진행 상황 확인하며 자율 작업
적대적 토론	두 에이전트가 반대 입장에서 토론하여 품질 향상 (GAN 원리)
활성화	`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` 설정 필요

> **비용 주의:** 일반 세션 대비 **7배** 토큰 소비. 에이전트 10명 x 15분 = 약 8만원. 대규모 작업(보안 감사, 코드 리팩토링)에만 사용 권장.

Git Worktree

Agent Team과 함께 사용할 때 효과적입니다. 각 Branch가 별도 작업 디렉토리를 가져서 **파일 충돌 없이 병렬 개발**이 가능합니다. 한 에이전트는 소개 페이지, 다른 에이전트는 연락처 페이지, 또 다른 에이전트는 서비스 페이지를 동시에 만들고 나중에 합칩니다.

전체 워크플로우 조합

CLAUDE.md → Plan Mode → 코드 작성 → 코드 리뷰 → 테스트 → 배포

반복 작업은 **Skill**로, 외부 연동은 **MCP/API Skill**로 자동화합니다. 이 워크플로우를 한번 구축해 놓으면 어떤 프로젝트든 동일한 품질 수준으로 빠르게 진행할 수 있습니다.

부록: 주요 키보드 단축키

단축키	설명
`Escape`	Claude 중지 (Ctrl+C가 아님!)
`Escape` x 2	이전 메시지 목록으로 점프
`Shift+Tab`	Plan Mode 전환
`Cmd+T`	Extended Thinking 토글
`Cmd+P`	모델 선택기
`Ctrl+G`	외부 에디터에서 편집

`Ctrl+O`	상세 모드(verbose) 토글
`Ctrl+B`	현재 작업을 백그라운드로 전환

부록: 오늘 당장 실행할 액션 아이템

1. **CLAUDE.md 만들기:** `/init`으로 시작. 자주 하는 실수나 반드시 따라야 할 규칙을 3줄만 추가해 보세요.
2. **Plan Mode 사용하기:** 다음 작업에서 바로 빌드하고 싶은 충동을 참고, 계획을 먼저 세워보세요.
3. **Skill 하나 만들기:** 매일 반복하는 작업 하나를 Skill로 만들어 보세요. 70% 정확도로 시작해서 점점 개선하면 됩니다.

부록: 보안 주의사항

> **반드시 기억하세요:**
>
> AI가 작성한 코드에는 보안 취약점이 숨어 있을 수 있습니다. 돈을 받고 서비스를 운영하기 전에는 반드시 보안 검토를 하셔야 합니다.
>
> 특히 **결제 기능**이나 **개인정보**를 다루는 서비스라면 전문가의 보안 리뷰를 꼭 받으세요.
>
> Claude Code는 엄청난 도구이지만, 모든 도구가 그렇듯 **제대로 알고 쓸 때 진가를 발휘합니다.**

Claude Code 핵심 가이드 | 2026년 2월 기준 | Opus 4.6

AI 도구는 빠르게 발전하고 있습니다. 최신 정보는 공식 문서를 참고하세요.

CLAUDE CODE

올인원 가이드북

설치 · 명령어 · 설정 · 실전 워크플로우

⌨️ CLI 명령어 | 🔧 MCP & 플러그인 | 🚀 실전 워크플로우

2026 년2 월 최신판 · Anthropic 공식 문서 기반

제작 · 배포: 투더제이 코딩클래스 (ttj.kr)

복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

C O N T E N T S

목차

01 **Claude Code 개요** 모델, 구독 플랜 3

02 **설치, 인증 및 첫 실행** curl 네이티브 설치, 인증, 첫 실행, 업데이트 4

03 **CLI 명령어 레퍼런스** 기본 실행 명령어, 주요 플래그, 시스템 프롬프트 7

04 **슬래시 명령어** 전체 내장 명령어, 커스텀 명령어, Frontmatter 9

05 **대화형 세션 단축키 & 기능** 파일 참조, 셸 실행, Plan 모드 15

06 **CLAUDE.md — 프로젝트 메모리** 계층 구조, 작성법, 자동 생성 16

07 **설정 파일 (settings.json)** 위치, 우선순위, 권한 관리 17

08 **Hooks — 자동화 트리거** 이벤트, 매처, 종료 코드, 실전 예시 7 가지 18

09 **MCP (Model Context Protocol)** 필수 MCP 서버 TOP 10, 설정 파일 26

10 **Plugins (플러그인)** 필수 플러그인, 마켓플레이스 29

11 **Skills — 확장 기능** 구조, Frontmatter, 호출 제어, 프로그레시브 디스클로저 31

12 **Sub-Agents (서브에이전트)** CLI 정의, 필드, 활용 패턴 34

13 **Agent Teams (실험적)** 멀티 에이전트 병렬 협업, 공유 태스크 36

14 **실전 워크플로우 & 베스트 프랙티스** Headless 모드, 프롬프팅 팁, 팀 협업 38

15 **Claude Code 내장 도구 목록** Read, Write, Edit, Bash 등 39

16 **참고 자료** 공식 문서, 커뮤니티 리소스 링크 40

CHAPTER 01

Claude Code 개요

Claude Code 는 Anthropic 이 개발한 터미널 기반 AI 코딩 에이전트입니다. 브라우저에서 대화하는 Claude.ai 와 달리, 개발자의 터미널에서 직접 실행되며 파일 시스템에 접근하여 코드를 읽고, 쓰고, 실행할 수 있습니다.

단순한 코드 자동완성이 아닌, 프로젝트 구조를 이해하고 코드를 작성·수정하며, 셸 명령을 실행하고, Git 작업을 관리하는 종합적인 코딩 파트너입니다.

1.1 사용 가능 모델

모델	특징	추천 용도
Sonnet 4.5	빠르고 안정적인 성능	일반적인 코딩 (기본값)
Haiku 4.5	가장 빠르고 토큰 절약	간단한 작업, 빠른 응답
Opus 4.5	최고 성능 (이전 세대 SOTA)	복잡한 멀티스텝 작업
Opus 4.6	최신 SOTA 모델	고난도 계획·추론 작업

1.2 구독 플랜

플랜	설명
Pro	기본 구독, Sonnet 4.5 중심 사용
Max5	Pro 대비 5 배 사용량, Opus 제한적 사용
Max20	최대 사용량, Opus 를 기본 모델로 사용
API	사용량 기반 과금, 토큰 제한 없음

ℹ Opus Plan Mode

Opus Plan Mode 는 연구·계획 단계에서 Opus 4.6 을, 구현·실행 단계에서 Sonnet 4.5 를 자동 전환합니다. 비용 효율과 품질을 동시에 잡을 수 있습니다.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 02

설치, 인증 및 첫 실행

2.1 네이티브 바이너리 설치 (권장)

Anthropic 공식 권장 설치 방법입니다. Node.js 가 필요하지 않으며, 자동 업데이트를 지원합니다.

macOS / Linux / WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows (PowerShell):

```
irm https://claude.ai/install.ps1 | iex
```

macOS Homebrew:

```
brew install --cask claude-code
```

> 💡 **특정 버전 설치**
> curl -fsSL https://claude.ai/install.sh | bash -s 2.0.22 처럼 버전을 지정할 수 있습니다.

> ⚠ **npm 설치는 deprecated**
> npm install -g @anthropic-ai/claude-code 방식은 더 이상 권장하지 않습니다. 기존 npm 설치를 네이티브로 마이그레이션하려면 claude install 을 실행하세요.

2.2 설치 확인

설치가 완료되면 다음 명령어로 확인합니다:

```
# 설치 확인
claude --version

# 상세 진단 (설치 타입, PATH, 권한 등)
claude doctor
```

claude doctor 결과에서 Config install method: native 가 표시되면 정상입니다.

2.3 인증 방법

Claude Code 를 처음 실행하면 인증을 요구합니다. 세 가지 방법 중 선택할 수 있습니다:

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

방법 1 — Claude Pro/Max 구독 (권장): claude 실행 시 브라우저가 열리며 claude.ai 계정으로 로그인합니다. 웹과 Claude Code 를 하나의 구독으로 사용합니다.

방법 2 — Anthropic Console (API): Console 에서 API 키를 발급받아 환경변수로 설정합니다. 사용량 기반 과금됩니다.

```
export ANTHROPIC_API_KEY=sk-ant-...
```

셸 프로파일(~/.bashrc, ~/.zshrc 등)에 추가하면 영구 적용됩니다.

방법 3 — Claude for Teams / Enterprise: 팀 관리자가 설정한 조직 계정으로 인증합니다. 중앙 집중식 빌링과 관리가 가능합니다.

2.4 첫 실행

인증이 완료되면 프로젝트 디렉토리에서 Claude Code 를 시작합니다:

```
# 프로젝트 디렉토리로 이동
cd /path/to/your/project

# Claude Code 시작 (대화형 REPL)
claude

# 또는 초기 프롬프트와 함께 시작
claude "이 프로젝트의 구조를 설명해줘"

# 마지막 대화 이어서 시작
claude -c
```

> **ℹ 첫 실행 시 안내**
> 처음 실행하면 텍스트 스타일(일반/간결/상세)을 선택하는 단계가 나옵니다. 이후 바로 대화형 세션이 시작됩니다. /help 명령어로 사용 가능한 명령어 목록을 확인하세요.

2.5 업데이트

네이티브 설치는 시작 시 자동으로 업데이트를 확인합니다. 수동 업데이트가 필요하면:

설치 방식	업데이트 명령어
네이티브 (curl)	자동 업데이트 (수동: claude update)
Homebrew	brew upgrade claude-code
Windows (WinGet)	winget upgrade Anthropic.ClaudeCode

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

2.6 VS Code / Cursor 확장

Claude Code 확장 프로그램을 설치하면 IDE 에서 인라인 diff 와 실시간 피드백으로 작업할 수 있습니다. 여러 창에서 병렬 실행도 가능합니다.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 03

CLI 명령어 레퍼런스

3.1 기본 실행 명령어

명령어	설명
`claude`	대화형 REPL 세션 시작
`claude "질문"`	초기 프롬프트와 함께 REPL 시작
`claude -p "질문"`	한 번 질의 후 종료 (스크립팅용)
`cat file | claude -p "질문"`	파이프로 파일 내용 전달 후 질의
`claude -c`	가장 최근 대화 이어서 계속하기
`claude -c -p "질문"`	최근 대화 이어서 SDK 모드로 질의
`claude -r "세션 ID"`	특정 세션 ID 로 대화 재개
`claude update`	최신 버전으로 업데이트
`claude mcp`	MCP 서버 설정 관리

💡 **세션 이어가기**

claude -c 를 사용하면 이전 컨텍스트를 유지한 채 작업을 계속할 수 있어, 장시간 리팩토링이나 단계적 기능 구현에 유용합니다.

3.2 주요 CLI 플래그

플래그	설명	예시
`--model`	세션에 사용할 모델 지정	claude --model opus
`--add-dir`	추가 작업 디렉토리 지정	claude --add-dir ../api
`--continue, -c`	최근 대화 이어하기	claude -c
`--resume, -r`	특정 세션 재개	claude -r abc123
`--print, -p`	비대화형 출력 모드	claude -p "질의"
`--output-format`	출력 포맷 (text/json/stream-json)	--output-format json
`--max-turns`	에이전트 턴 수 제한	--max-turns 3

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

플래그	설명	예시
`--allowedTools`	허용 도구 목록	Read Write Bash(git *)
`--disallowedTools`	차단 도구 목록	Edit
`--verbose`	상세 로그 출력	claude --verbose
`--debug`	디버그 모드	--debug "api,mcp"
`--version, -v`	버전 확인	claude -v
`--mcp-config`	MCP 설정 파일 로드	--mcp-config ./mcp.json
`--permission-mode`	권한 모드 지정	--permission-mode plan
`--dangerously-skip-permissions`	권한 프롬프트 건너뛰기	(주의하여 사용)
`--fallback-model`	모델 과부하 시 대체	--fallback-model sonnet
`--agents`	커스텀 서브에이전트 (JSON)	(JSON 객체)
`--fork-session`	세션 재개 시 새 세션	--resume abc --fork
`--json-schema`	구조화된 JSON 출력	(JSON Schema)

3.3 시스템 프롬프트 커스터마이징

플래그	동작	사용 시기
`--system-prompt`	기본 프롬프트 전체 교체	완전한 제어가 필요할 때
`--system-prompt-file`	파일에서 프롬프트 로드	버전 관리 (print 모드)
`--append-system-prompt`	기본 프롬프트에 추가	기본 기능 유지+추가 지시
`--append-system-prompt-file`	파일에서 추가 로드	팀 공유 추가 지시

> ✅ **권장**
> --append-system-prompt 를 사용하면 Claude Code 의 기본 능력을 유지하면서 추가 지시를 할 수 있어 가장 안전합니다.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 04

슬래시 명령어 (Slash Commands)

Claude Code 세션 내에서 / 접두사로 실행하는 명령어입니다. /help 를 입력하면 전체 목록을 확인할 수 있습니다. 슬래시 명령어는 Claude 의 대화 모드를 거치지 않고 즉시 실행되므로 토큰을 절약할 수 있습니다.

4.1 전체 내장 슬래시 명령어

📋 핵심 명령어 — 매일 사용

명령어	설명
`/help`	사용 가능한 모든 슬래시 명령어 목록 표시
`/clear`	대화 기록 초기화 (토큰 절약, 새 작업 시작 시 필수)
`/compact [지시]`	대화를 요약하여 컨텍스트 압축. 선택적으로 초점 지시 가능
`/model [모델명]`	모델 변경. /model sonnet, /model opus, /model haiku
`/exit`	세션 종료 (Ctrl+C 2 회와 동일)

🚀 프로젝트 초기화 & 설정

명령어	설명
`/init`	CLAUDE.md 파일 자동 생성 (새 프로젝트 시작 시 가장 먼저 실행)
`/config`	설정 인터페이스 열기 (Config 탭)
`/permissions`	권한 설정 확인/수정 (allowedTools, deny 등)
`/memory`	CLAUDE.md 메모리 파일 에디터 열기
`/add-dir`	세션 중 추가 작업 디렉토리 연결 (멀티 레포 작업)

📊 상태 확인 & 모니터링

명령어	설명
`/status`	설정 상태 표시 — 버전, 모델, 계정, 연결 상태
`/context`	컨텍스트 윈도우 사용량 시각화 (컬러 그리드)
`/cost`	토큰 사용량 및 비용 통계 표시

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

명령어	설명
/usage	플랜 사용 한도 및 레이트 리밋 상태 (구독자 전용)
/doctor	설치 상태 진단 (PATH, 버전, 권한 등)
/todos	현재 세션의 TODO 항목 목록 표시

🔧 도구 & 확장 관리

명령어	설명
/hooks	Hook 설정 대화형 관리 (추가/수정/삭제)
/mcp	MCP 서버 연결 및 OAuth 인증 관리
/plugin	플러그인 설치/목록/관리
/agents	커스텀 AI 서브에이전트 관리
/ide	IDE 연동 관리 및 상태 확인
/sandbox	샌드박스 모드 활성화 (파일시스템/네트워크 격리)

📝 코드 리뷰 & Git

명령어	설명
/review	최근 변경 사항에 대한 코드 리뷰 요청
/security-review	현재 브랜치 변경사항 보안 검토
/pr-comments	GitHub PR 코멘트 확인
/install-github-app	Claude GitHub Actions 앱 설치

⏪ 대화 관리

명령어	설명
/resume	이전 대화 이어서 시작
/rewind	대화 및/또는 코드를 이전 상태로 되돌리기
/export [파일명]	현재 대화를 파일이나 클립보드로 내보내기
/bashes	백그라운드 작업 목록 확인 및 관리

⚙ 표시 & 인터페이스

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

명령어	설명
`/output-style [스타일]`	응답 스타일 설정 (concise, detailed 등)
`/statusline`	상태 표시줄 UI 설정
`/terminal-setup`	Shift+Enter 줄바꿈 키 바인딩 설치 (iTerm2/VSCode)
`/vim`	Vim 모드 진입 (insert/command 모드 전환)

🔐 계정 & 기타

명령어	설명
`/login`	Anthropic 계정 전환
`/logout`	Anthropic 계정 로그아웃
`/privacy-settings`	개인정보 설정 확인/수정
`/release-notes`	릴리스 노트 확인
`/bug`	버그 리포트 (대화 내용을 Anthropic 에 전송)

> 💡 **/clear 습관화**
> 새로운 작업을 시작할 때마다 /clear 로 대화를 초기화하세요. 불필요한 기록이 토큰을 소모하고, 컨텍스트 압축 호출을 유발합니다.

> ℹ **/compact 사용 팁**
> /compact 뒤에 초점을 지정할 수 있습니다. 예: /compact 인증 모듈 관련 내용만 유지. Claude 가 지정된 내용을 우선 보존하며 나머지를 요약합니다.

4.2 자주 쓰는 명령어 조합

실전에서 효과적인 명령어 사용 패턴입니다:

```
# 새 작업 시작 시 추천 순서
/clear                          ← 이전 대화 정리
/init                           ← CLAUDE.md 없다면 생성
/model opus                     ← 복잡한 작업이면 Opus 로

# 대화가 길어졌을 때
/context                        ← 현재 사용량 확인
/compact 핵심 로직만 유지        ← 컨텍스트 압축

# 코드 리뷰 워크플로우
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

```
/review                          ← 전체 리뷰
/security-review                 ← 보안 검토 추가
/pr-comments                     ← PR 코멘트 확인

# 비용 확인
/cost                            ← 이번 세션 토큰 비용
/usage                           ← 플랜 한도 확인
```

4.3 커스텀 슬래시 명령어 만들기

마크다운 파일로 나만의 슬래시 명령어를 만들 수 있습니다. 자주 반복하는 프롬프트를 명령어로 저장하면 토큰도 절약되고 일관성도 유지됩니다.

저장 위치	적용 범위
`.claude/commands/`	프로젝트 전용 (팀과 git 으로 공유 가능)
`~/.claude/commands/`	모든 프로젝트 공용 (개인 전역)

예시 1: 프로젝트 최적화 명령어

```
mkdir -p .claude/commands
cat > .claude/commands/optimize.md << 'EOF'
코드의 성능 문제를 분석하고 최적화를 제안해주세요:
1. 불필요한 렌더링이나 재계산 확인
2. N+1 쿼리 패턴 검사
3. 메모리 누수 가능성 확인
4. 개선안을 코드와 함께 제시
EOF

# 사용: /optimize
```

예시 2: $ARGUMENTS 로 동적 입력 받기

```
cat > .claude/commands/fix-issue.md << 'EOF'
이슈 #$ARGUMENTS 를 분석하고 수정해주세요.
코딩 표준을 준수하고, 테스트도 추가해주세요.
EOF

# 사용: /fix-issue 123
# $ARGUMENTS → "123"
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

예시 3: 개별 인자 $1, $2, $3

```
cat > .claude/commands/review-pr.md << 'EOF'
PR #$1 을(를) 리뷰해주세요.
우선순위: $2
담당자: $3
EOF

# 사용: /review-pr 456 high alice
# $1→"456", $2→"high", $3→"alice"
```

예시 4: Bash 명령 실행 (! 접두사)

커스텀 명령어 안에서 !`명령어` 구문으로 셸 명령을 실행하고 결과를 컨텍스트에 포함시킬 수 있습니다:

```
---
allowed-tools: Bash(git add:*), Bash(git status:*), Bash(git commit:*)
description: 변경사항을 자동 커밋
---

## 현재 상태
- Git 상태: !`git status`
- 변경 내용: !`git diff HEAD`
- 현재 브랜치: !`git branch --show-current`

## 작업
위 변경사항을 분석하여 적절한 커밋 메시지로 커밋해주세요.
```

예시 5: 파일 참조 (@ 접두사)

```
# 커스텀 명령어 내에서 @ 접두사로 파일 참조
@src/utils/helpers.js 의 구현을 리뷰해주세요.
@src/old.js 와 @src/new.js 를 비교 분석해주세요.
```

4.4 커스텀 명령어 Frontmatter

명령어 파일 상단에 YAML frontmatter 로 메타데이터를 지정할 수 있습니다:

필드	설명
`allowed-tools`	명령어가 사용할 수 있는 도구 목록
`description`	명령어 설명 (/help 에 표시)
`argument-hint`	인자 힌트 (자동완성 시 표시). 예: add [id] | remove [id]
`model`	이 명령어에서 사용할 모델 지정

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

필드	설명
`disable-model-invocation`	Claude 가 자동으로 이 명령어를 호출하지 못하게 설정
`user-invocable`	사용자만 /명령어로 호출 가능 (Claude 자동 호출 차단)

4.5 네임스페이싱 (하위 폴더 정리)

명령어 파일을 하위 폴더로 정리하면 /help 에서 출처가 표시됩니다:

```
.claude/commands/frontend/component.md
→ /component (project:frontend)

~/.claude/commands/review.md
→ /review (user)
```

명령어 이름은 파일명에서 결정되며, 폴더는 분류용으로만 사용됩니다.

4.6 MCP 슬래시 명령어

MCP 서버가 제공하는 도구도 슬래시 명령어로 사용 가능합니다:

```
# MCP 서버의 도구를 슬래시 명령어로 실행
/mcp__github__search_repositories query="claude code"
/mcp__filesystem__read_file path="./config.json"
```

MCP 서버의 이름 규칙: mcp__<서버이름>__<도구이름>

> ✅ **Skills 과의 관계**
> 커스텀 슬래시 명령어(.claude/commands/)와 Skills(.claude/skills/)은 동일한 /명령어로 동작합니다. 기존 commands/ 파일은 그대로 유지됩니다. Skills 는 SKILL.md + scripts/ + references/ 구조로 더 강력한 기능을 제공합니다.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 05

대화형 세션 단축키 & 기능

키 / 기능	설명
@파일경로	파일이나 디렉토리를 프롬프트에 참조
! 셸명령어	Claude 를 거치지 않고 직접 셸 명령 실행
Shift+Tab	Plan 모드 전환 (코드 작성 전 계획)
Ctrl+C	현재 응답 중단
Esc	응답 생성 취소
/model opus	세션 중 모델 변경

파일 참조 예시:

```
> 이 코드를 분석해주세요 @./src/user-service.js
> @./src/ 폴더의 전체 구조를 설명해주세요
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 06

CLAUDE.md — 프로젝트 메모리

CLAUDE.md 는 Claude Code 의 "메모리" 역할을 하는 마크다운 파일입니다. 프로젝트의 코딩 표준, 아키텍처, 자주 쓰는 명령어를 기록하면 매 세션마다 자동으로 참조합니다.

6.1 계층 구조

위치	범위	용도
`~/.claude/CLAUDE.md`	모든 프로젝트 공용	개인 선호사항, 코딩 스타일
`./CLAUDE.md`	해당 프로젝트 전용	아키텍처, 빌드/린트 명령어
`./src/CLAUDE.md`	해당 폴더 전용	모듈별 세부 규칙

6.2 CLAUDE.md 작성 예시

```
# 프로젝트: My App

## 기술 스택
- React + TypeScript + Tailwind CSS
- Node.js + Express 백엔드

## 주요 명령어
- 빌드: npm run build
- 테스트: npm test
- 린트: npm run lint

## 코딩 규칙
- 함수형 컴포넌트 사용
- camelCase 변수명
- 모든 API 호출에 에러 핸들링 필수
```

> 💡 **Git 에 커밋하기**
>
> CLAUDE.md 와 .claude/ 디렉토리를 Git 에 커밋하면 팀원 모두가 동일한 설정을 공유합니다. /init 으로 초기 파일을 자동 생성할 수 있습니다.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 07

설정 파일 (settings.json)

7.1 설정 파일 위치 및 우선순위

레벨	위치	용도
Enterprise	관리자 배포	조직 전체 정책 강제
Local	.claude/settings.local.json	개인 로컬 (Git 제외)
Project	.claude/settings.json	팀 공유 프로젝트 설정
User	~/.claude/settings.json	전역 개인 기본값

우선순위: Enterprise > Local > Project > User

7.2 설정 예시

```
{
  "model": "claude-sonnet-4-5-20250929",
  "maxTokens": 4096,
  "permissions": {
    "allowedTools": ["Read", "Write", "Bash(git *)"],
    "deny": [
      "Read(./.env)",
      "Read(./.env.*)",
      "Write(./production.config.*)"
    ]
  }
}
```

⚠ **보안 주의**

allowedTools 로 자동 승인할 도구를 지정하고, deny 로 .env 나 프로덕션 설정 파일을 차단하세요. 민감한 파일 접근을 방지할 수 있습니다.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 08

Hooks — 자동화 트리거

Hooks 는 Claude Code 가 특정 동작을 할 때 자동으로 실행되는 나만의 명령어입니다. 예를 들어, Claude 가 파일을 수정할 때마다 자동으로 코드 포맷팅을 하거나, 위험한 명령어를 차단할 수 있습니다.

> **ℹ Hooks 를 쉽게 이해하기**
> Hooks 는 "~하면 자동으로 ~해라"라는 규칙입니다. 예: "파이썬 파일을 저장하면 → 자동으로 Black 포맷터를 실행해라". 프롬프트로 매번 부탁하는 것과 달리, Hooks 는 한 번 설정하면 매번 빠짐없이 실행됩니다.

8.1 프롬프트 vs Hooks — 왜 Hooks 인가?

비교	프롬프트로 부탁하기	Hooks 설정하기
방식	"파일 수정 후 Black 실행해줘"	settings.json 에 한 번 설정
실행 보장	Claude 가 잊을 수 있음	100% 매번 자동 실행
토큰 비용	매번 프롬프트에 토큰 소비	토큰 비용 없음 (셸 명령)
일관성	세션마다 다시 부탁해야 함	설정 한 번이면 영구 적용

8.2 Hooks 의 동작 흐름

Hooks 는 Claude 의 작업 과정 중 특정 시점에 끼어들어 실행됩니다. 아래 흐름으로 이해하세요:

```
사용자 프롬프트 입력
    ↓
  [UserPromptSubmit Hook] ← 프롬프트에 컨텍스트 추가 가능
    ↓
Claude 가 도구 사용 결정 (예: 파일 쓰기)
    ↓
  [PreToolUse Hook] ← 실행 전 검사 / 차단 가능
    ↓
  [PermissionRequest Hook] ← 권한 자동 승인 가능
    ↓
도구 실행 (파일 쓰기 완료)
    ↓
  [PostToolUse Hook] ← 포맷팅, 린팅, 테스트 실행
    ↓
Claude 응답 완료
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

```
    ↓
  [Stop Hook] ← 결과 검증, git 커밋
```

8.3 Hook 이벤트 종류

각 이벤트는 "언제 실행할 것인가"를 결정합니다:

이벤트	한 줄 설명	대표 활용 예
`PreToolUse`	도구 실행 직전	.env 수정 차단, rm -rf 차단
`PostToolUse`	도구 실행 직후	Black/Prettier 포맷팅, 테스트
`PermissionRequest`	"허용?" 묻기 직전	npm test 자동 승인
`UserPromptSubmit`	프롬프트 제출 시	스프린트 목표 자동 첨부
`SessionStart`	세션 시작 시	git status, TODO 로드
`SessionEnd`	세션 종료 시	로그 저장, 임시 파일 정리
`Stop`	Claude 응답 완료 시	결과 검증, 브랜치 푸시
`Notification`	알림 발생 시	데스크톱/Slack 알림
`SubagentStop`	서브에이전트 완료 시	서브에이전트 결과 로깅
`PreCompact`	/compact 실행 전	요약 전 데이터 백업

8.4 나의 첫 Hook 만들기 (단계별)

가장 쉬운 방법인 /hooks 명령어로 만들어 봅시다. TypeScript 파일 저장 시 자동으로 Prettier 를 실행하는 Hook 입니다.

Step 1 — Claude Code 세션에서 /hooks 를 입력합니다.
Step 2 — "Add hook" 선택 → 이벤트로 "PostToolUse" 선택
Step 3 — 매처에 Write(*.ts) 입력 (TS 파일 쓰기에만 반응)
Step 4 — 명령어에 npx prettier --write $file 입력
Step 5 — 저장! 이제 Claude 가 .ts 파일을 수정할 때마다 자동으로 Prettier 가 실행됩니다.

> 💡 **/hooks 메뉴**
> /hooks 명령어는 대화형 메뉴를 제공합니다. JSON 을 직접 수정하지 않아도 쉽게 Hook 을 추가, 수정,

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

> 삭제할 수 있습니다.

8.5 JSON 으로 직접 설정하기

더 세밀한 제어가 필요하면 settings.json 에 직접 작성합니다:

설정 파일 위치	적용 범위
`~/.claude/settings.json`	모든 프로젝트 (전역)
`.claude/settings.json`	해당 프로젝트만 (팀 공유 가능)
`.claude/settings.local.json`	해당 프로젝트, 개인만 (git 제외)

기본 구조는 이렇습니다:

```
{
  "hooks": {
    "이벤트이름": [        ← 언제?
      {
        "matcher": "도구",  ← 어떤 도구에서?
        "hooks": [
          {
            "type": "command",
            "command": "명령어"  ← 무엇을 실행?
          }
        ]
      }
    ]
  }
}
```

8.6 매처(Matcher) — 어떤 도구에서 실행할까?

매처로 Hook 이 반응할 도구를 필터링합니다:

패턴	의미
`"Write"`	Write 도구에서만 실행
`"Edit|Write"`	Edit 또는 Write 에서 실행 (| 로 복수 지정)
`"Edit|MultiEdit|Write"`	모든 파일 수정 도구에서 실행
`"*"` 또는 `""`	모든 도구에서 실행 (주의: 느려질 수 있음)

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

패턴	의미
`"Write(*.py)"`	.py 파일을 Write 할 때만
`"Bash(npm test*)"`	npm test 로 시작하는 Bash 명령만
`"mcp__github__.*"`	GitHub MCP 서버의 모든 도구

> ⚠ **대소문자 주의**
> 매처는 대소문자를 구분합니다. "bash"는 "Bash" 도구와 매칭되지 않습니다. Claude Code 의 도구 이름은 Read, Write, Edit, MultiEdit, Bash, Glob, Grep 등입니다.

8.7 종료 코드 — Hook 으로 Claude 제어하기

Hook 스크립트가 끝날 때 반환하는 숫자(종료 코드)로 Claude 의 행동을 결정합니다:

종료 코드	의미	Claude 에게 미치는 영향
`0`	성공 ✅	정상 진행, 아무 문제 없음
`2`	차단 🚫	도구 실행을 취소하고, stderr 메시지를 Claude 에게 피드백
`그 외 (1 등)`	오류 ⚠	Hook 오류로 기록되지만, Claude 작업은 계속됨

> ℹ **exit 2 의 강력함**
> PreToolUse 에서 exit 2 를 반환하면 Claude 는 해당 도구 사용을 취소합니다. stderr 로 출력한 메시지가 Claude 에게 전달되어, Claude 가 이유를 이해하고 다른 방법을 시도합니다.

8.8 실전 예시 — 복사해서 바로 쓰기

📝 예시 1: Python 파일 자동 포맷팅 (가장 기본)

Claude 가 .py 파일을 수정하면 Black 포맷터와 mypy 타입 체커를 자동 실행합니다:

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write(*.py)",
      "hooks": [
        { "type": "command", "command": "python -m black $file" },
        { "type": "command", "command": "mypy $file" }
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

```
      ]
    }]
  }
}
```

💅 예시 2: TypeScript/JavaScript 자동 포맷팅 + 린팅

프론트엔드 프로젝트에서 Prettier 와 ESLint 를 자동 실행합니다:

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.file_path' | { read f; if echo \"$f\" |
grep -q '\\.ts$\\|\\.tsx$'; then npx prettier --write \"$f\" && npx eslint --
fix \"$f\"; fi; }"
      }]
    }]
  }
}
```

🛡️ 예시 3: .env / .git 파일 보호

민감한 파일을 Claude 가 수정하지 못하도록 차단합니다:

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "python3 -c \"import json,sys; d=json.load(sys.stdin);
p=d.get('tool_input',{}).get('file_path',''); bad=['.env','.git/','package-
lock']; sys.exit(2 if any(x in p for x in bad) else 0)\""
      }]
    }]
  }
}
```

→ Claude 가 .env 를 수정하려 하면 exit 2 로 차단되고, Claude 는 이유를 이해하여 다른 방법을 제안합니다.

⛔ 예시 4: 위험한 Bash 명령 차단

rm -rf 같은 위험한 명령을 사전에 차단합니다:

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

```
        "type": "command",
        "command": "jq -r '.tool_input.command' | { read cmd; if echo \"$cmd\"
| grep -qE 'rm\\s+-rf|drop\\s+database|format\\s+'; then echo '위험한 명령 차단됨'
>&2; exit 2; fi; }"
      }]
    }]
  }
}
```

✅ 예시 5: npm test 자동 승인 (권한 팝업 제거)

매번 "허용하시겠습니까?" 묻는 팝업을 자동 승인합니다:

```
{
  "hooks": {
    "PermissionRequest": [{
      "matcher": "Bash(npm test*)",
      "hooks": [{
        "type": "command",
        "command": "echo '{\"decision\":{\"behavior\":\"allow\"}}'"
      }]
    }]
  }
}
```

🔔 예시 6: 데스크톱 알림 (작업 완료/입력 대기)

```
// macOS
"command": "osascript -e 'display notification \"입력이 필요합니다\" with
title \"Claude Code\"'"

// Linux
"command": "notify-send 'Claude Code' '입력이 필요합니다'"
```

🚀 예시 7: 세션 시작 시 프로젝트 상태 자동 로드

세션이 시작되면 git 상태와 최근 TODO 를 자동으로 Claude 에게 전달합니다:

```
{
  "hooks": {
    "SessionStart": [{
      "hooks": [{
        "type": "command",
        "command": "echo \"=== Git Status ===\" && git status --short &&
echo \"=== Recent TODOs ===\" && grep -rn 'TODO:' src/ --include='*.ts' | head
-10"
      }]
    }]
  }
}
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

8.9 여러 Hook 조합하기

하나의 이벤트에 여러 매처를 등록하고, 하나의 매처에 여러 명령을 연결할 수 있습니다:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write(*.py)",
        "hooks": [
          { "type": "command", "command": "python -m black $file" },
          { "type": "command", "command": "mypy $file" }
        ]
      },
      {
        "matcher": "Write(*.ts)",
        "hooks": [
          { "type": "command", "command": "npx prettier --write $file" }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "./scripts/block-env.sh" }
        ]
      }
    ]
  }
}
```

8.10 디버깅 & 문제 해결

Hook 이 예상대로 작동하지 않을 때 확인할 사항:

Hook 이 실행되지 않나요? /hooks 에서 올바른 이벤트에 등록되어 있는지 확인하세요. 매처 대소문자가 정확한지 확인하세요 (Write ≠ write).

오류 메시지를 보고 싶다면? Ctrl+O 로 verbose 모드를 켜면 stderr 출력이 표시됩니다. 디버깅 시 echo "디버그 메시지" >&2 를 추가해 보세요.

Hook 타임아웃: Hook 실행 제한 시간은 10 분입니다. 그 이상 걸리는 작업은 백그라운드로 처리하세요.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

8.11 주의사항

⚠ **성능 영향**

매처를 "*"로 설정하면 모든 도구 호출마다 Hook 이 실행되어 작업이 크게 느려집니다. 반드시 필요한 도구에만 매칭하세요. Hook 은 토큰 비용은 없지만 실행 시간이 누적됩니다.

✅ **초심자 추천 순서**

① PostToolUse + Write(*.py) → 자동 포맷팅부터 시작하세요. ② PreToolUse + Edit|Write → .env 파일 보호를 추가하세요. ③ Notification → 작업 완료 알림을 받으세요. ④ SessionStart → 컨텍스트 자동 로드로 발전하세요.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 09

MCP (Model Context Protocol)

MCP 는 Claude Code 에 외부 도구와 서비스를 연결하는 오픈 프로토콜입니다. GitHub, 데이터베이스, 브라우저 자동화, Notion 등 다양한 통합이 가능하며, Claude 를 단순 코딩 도구에서 풀스택 자동화 에이전트로 확장합니다.

9.1 MCP 설정 방법

```
# 서버 추가 (stdio 방식)
claude mcp add <이름> -- <명령어>

# HTTP 방식 (원격 서버)
claude mcp add --transport http <이름> <URL>

# 설정 확인
claude mcp list

# JSON 파일에서 일괄 로드
claude --mcp-config ./mcp.json
```

9.2 필수 MCP 서버 TOP 10

가장 많이 사용되고 개발 생산성을 크게 높이는 MCP 서버들입니다:

① GitHub MCP — 코드 저장소 관리

이슈, PR, CI/CD, 커밋 분석을 터미널에서 직접 수행합니다. 가장 많이 사용되는 필수 MCP 입니다.

```
claude mcp add github -- npx -y @modelcontextprotocol/server-github

# 환경변수 설정 필요
export GITHUB_PERSONAL_ACCESS_TOKEN=ghp_xxxxx
```

활용 예: "인증 관련 이슈를 모두 찾아줘", "feature 브랜치에서 PR 을 만들어줘"

② Context7 — 최신 라이브러리 문서 접근

AI 의 학습 데이터가 아닌, 실시간 최신 공식 문서를 가져와 정확한 코드를 생성합니다. 할루시네이션을 크게 줄여줍니다.

```
claude mcp add context7 -- npx -y @upstash/context7-mcp@latest
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

활용 예: "최신 Next.js 14 Server Components 패턴으로 만들어줘 - use context7"

③ Filesystem — 파일 시스템 제어

세밀한 파일 시스템 접근 권한을 제어하며, 복잡한 코드베이스 리팩토링에 유용합니다.

```
claude mcp add filesystem -- npx -y @modelcontextprotocol/server-filesystem
/path/to/dir
```

④ PostgreSQL — 데이터베이스 직접 질의

데이터 분석, DB 튜닝, 백엔드 개발 시 SQL 쿼리를 직접 실행합니다.

```
claude mcp add postgres -- npx -y @modelcontextprotocol/server-postgres
postgresql://user:pass@localhost/db
```

활용 예: "비활성 사용자를 모두 찾아서 CSV 로 내보내줘"

⑤ Playwright — 브라우저 자동화 & 테스트

웹 애플리케이션 E2E 테스트, 폼 작성, 스크래핑을 자동화합니다. 접근성 트리 기반으로 안정적입니다.

```
claude mcp add playwright -- npx -y @anthropic-ai/mcp-server-playwright
```

⑥ Sequential Thinking — 구조화된 사고

복잡한 문제를 단계별로 분석하는 반성적 사고 프로세스를 제공합니다. 아키텍처 결정이나 디버깅에 효과적입니다.

```
claude mcp add thinking -- npx -y @modelcontextprotocol/server-sequential-
thinking
```

⑦ Brave Search — 웹 검색

개발 중 실시간 웹 검색으로 최신 정보를 찾습니다. API 키가 필요합니다.

```
claude mcp add brave-search -- npx -y @modelcontextprotocol/server-brave-
search
# BRAVE_API_KEY 환경변수 필요
```

⑧ Notion — 지식 베이스 통합

Notion 페이지와 데이터베이스를 읽고 업데이트하여 지식 관리와 코드 개발을 연결합니다.

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

⑨ Sentry — 에러 모니터링

프로덕션 에러 보고서를 분석하고 즉시 수정 제안을 받을 수 있습니다.

설치: SSE 원격 서버 방식, Sentry API 토큰 필요

⑩ Docker — 컨테이너 관리

컨테이너 빌드, 실행, 로그 확인을 AI 명령으로 직접 관리합니다.

활용 예: "Nginx 컨테이너가 왜 시작하지 않는지 확인하고 고쳐줘"

> ⚠ **컨텍스트 관리 주의**
> MCP 서버를 너무 많이 활성화하면 200K 컨텍스트 창이 70K 까지 줄어들 수 있습니다. 프로젝트당 10 개 이하, 활성 도구 80 개 이하를 유지하고, 프로젝트별로 필요한 것만 활성화하세요.

9.3 MCP 설정 파일 예시 (mcp.json)

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_xxx" }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": { "POSTGRES_CONNECTION_STRING": "postgresql://..." }
    }
  }
}
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 10

Plugins (플러그인)

2025 년 12 월 Anthropic 이 공식 플러그인 마켓플레이스를 런칭했습니다. 현재 43 개 이상의 마켓플레이스에 834 개 이상의 플러그인이 등록되어 있으며, 커뮤니티가 빠르게 성장하고 있습니다.

10.1 플러그인 기본 사용법

```
# 플러그인 매니저 열기
/plugin

# 공식 마켓플레이스에서 설치
/plugin install <플러그인명>@claude-plugin-directory

# 설치된 플러그인 확인
/plugin  →  Installed 탭

# 외부 마켓플레이스 추가
/plugin marketplace add <소유자>/<저장소>
```

10.2 추천 필수 플러그인

플러그인	설명
LSP (TypeScript)	코드 점프, 참조 찾기, 타입 에러 실시간 확인 (공식)
LSP (Python)	Python Language Server 연결 (공식)
Context7	최신 라이브러리 문서를 프롬프트에 자동 주입
Ralph Loop	자율 코딩 세션 — CRUD, 마이그레이션 등 반복 작업 자동화
Chrome DevTools MCP	Chrome DevTools 로 네트워크 디버깅, 콘솔 에러 분석
commit-commands	Git 워크플로우 커밋/PR 자동화 명령어 (공식 데모)
Supermemory	Claude 가 실시간 학습하고 지식을 업데이트
Fullstack Dev Skills	65 개 풀스택 개발 스킬, Jira/Confluence 통합
chrome-devtools-mcp	라이브 페이지 디버깅, LCP 분석, 네트워크 검사
cc-usage	토큰 소비량 추적 및 비용 최적화 유틸리티

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

10.3 인기 커뮤니티 마켓플레이스

마켓플레이스	특징
`claude-plugins-official`	Anthropic 공식 (자동 등록)
`claude-code-plugins`	Anthropic 데모 플러그인
`paddo/claude-tools`	Gemini, Codex, Headless 등 6 개 특화 플러그인
`wshobson/commands`	57 개 프로덕션 슬래시 명령어
`awesome-claude-code`	커뮤니티 큐레이션 리소스 모음

> **ℹ 설치 범위 선택**
> 플러그인 설치 시 User(모든 프로젝트), Project(팀 공유), Local(개인 전용) 범위를 선택할 수 있습니다. /plugin → Discover 탭에서 Enter 를 눌러 범위를 지정하세요.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 11

Skills — 확장 기능

Skills 는 Claude 의 능력을 확장하는 모듈형 지식 패키지입니다. SKILL.md 파일과 선택적 스크립트, 리소스로 구성되며, 커스텀 슬래시 명령어와 통합됩니다. Claude 가 컨텍스트에 따라 자동으로 로드하거나, 사용자가 /skill-name 으로 직접 호출할 수 있습니다.

> **ℹ Skills vs Commands**
> 기존 .claude/commands/ 파일은 Skills 로 통합되었습니다. 기존 파일도 계속 동작하며, Skills 는 Frontmatter 제어, 서브에이전트 실행, 자동 호출 등 추가 기능을 제공합니다.

11.1 프로그레시브 디스클로저 (Progressive Disclosure)

Skills 의 핵심 설계 원리입니다. 시작 시 모든 스킬의 name/description 만 시스템 프롬프트에 로드합니다(1 단계). 사용자 요청과 관련되면 SKILL.md 전문을 로드합니다(2 단계). 필요한 참조 파일만 추가 로드합니다(3 단계). 이 방식으로 컨텍스트 윈도우를 효율적으로 사용합니다.

11.2 Skills 디렉토리 구조

```
my-skill/
  SKILL.md          # 메인 지침서 (필수)
  scripts/          # 실행 가능한 코드 (Python/Bash)
  references/       # 컨텍스트에 로드할 참조 문서
  assets/           # 템플릿, 아이콘, 폰트 등 출력물
```

11.3 Skills 저장 위치

위치	범위	용도
~/.claude/skills/	모든 프로젝트 공용	개인 전역 스킬
.claude/skills/	해당 프로젝트 전용	프로젝트 팀 공유 스킬
플러그인 제공	플러그인에 번들	마켓플레이스에서 설치

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

11.4 Frontmatter 필드

필드	설명
`name (필수)`	스킬 이름, /slash-command 가 됨
`description (필수)`	자동 호출 트리거 — '무엇을 하고 언제 쓰는지' 포함
`disable-model-invocation`	true 면 사용자만 호출 가능 (deploy, commit 등)
`user-invocable`	false 면 Claude 만 자동 호출 (배경 지식용)
`context: fork`	서브에이전트로 독립 실행 (대화 기록 미상속)
`allowed-tools`	스킬 실행 시 허용 도구 제한
`model`	스킬 실행 시 사용할 모델 오버라이드

11.5 SKILL.md 작성 예시

```
---
name: pr-review
description: PR 코드 리뷰 수행. 코드 리뷰, PR 검토,
  머지 리퀘스트 감사 요청 시 사용.
disable-model-invocation: true
---

# Pull Request 리뷰

## 워크플로우
1. gh pr view로 PR 정보 가져오기
2. 보안 취약점 검사 (OWASP Top 10)
3. 로직 오류 및 엣지 케이스 확인
4. 에러 핸들링 검증
5. 심각도별 정리: Critical > High > Medium > Low
```

> 💡 **description 이 핵심**
> description 은 자동 호출의 트리거 메커니즘입니다. '무엇을 하는지'와 '언제 쓰는지'를 모두 포함하세요. 본문의 'When to Use' 섹션은 이미 로드된 후에만 보이므로 트리거에 도움이 되지 않습니다.

11.6 스킬 작성 베스트 프랙티스

SKILL.md 는 500 줄 이하로: 상세 내용은 references/ 폴더의 별도 파일로 분리하세요.

컨텍스트 비용 의식: Claude 가 이미 아는 것은 생략하고, 모르는 것만 추가하세요.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

스크립트 활용: scripts/ 폴더의 코드는 실행 결과만 컨텍스트에 들어가므로 토큰을 절약합니다.

신뢰할 수 있는 소스만 사용: 악성 스킬은 데이터 유출 등의 위험이 있으므로, 직접 만들었거나 Anthropic 공식 스킬만 사용하세요.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 12

Sub-Agents (서브에이전트)

서브에이전트는 메인 세션 내에서 실행되는 특정 도메인 전문 Claude 인스턴스입니다. 결과를 메인 에이전트에게만 보고하며, 서로 간 직접 통신은 불가능합니다. 복잡한 작업을 전문 영역별로 분업할 때 효과적입니다.

12.1 서브에이전트 vs 에이전트 팀

비교	설명
서브에이전트	단일 세션 내 실행, 메인에게만 보고, 직접 통신 불가
에이전트 팀	독립 세션, 공유 태스크 리스트, 팀원 간 직접 메시징

서브에이전트는 순차적·종속적 작업에, 에이전트 팀은 병렬·독립적 작업에 적합합니다.

12.2 CLI 에서 서브에이전트 정의

```
claude --agents '{
  "code-reviewer": {
    "description": "코드 변경 후 자동으로 리뷰합니다.",
    "prompt": "시니어 코드 리뷰어. 품질, 보안, 모범 사례 집중.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  },
  "debugger": {
    "description": "오류와 테스트 실패를 전문 디버깅합니다.",
    "prompt": "전문 디버거. 오류 분석, 근본 원인, 수정안 제시.",
    "model": "opus"
  },
  "test-writer": {
    "description": "테스트 코드를 작성합니다.",
    "prompt": "TDD 전문가. 단위·통합 테스트 작성.",
    "tools": ["Read", "Write", "Bash"]
  }
}'
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

12.3 필드 설명

필드	설명
`description (필수)`	에이전트 호출 조건을 설명하는 자연어
`prompt (필수)`	에이전트 행동을 안내하는 시스템 프롬프트
`tools (선택)`	사용 가능한 도구 배열 (기본: 전체 상속)
`model (선택)`	모델 별칭 (sonnet, opus, haiku)

12.4 Skills 에서 서브에이전트 실행

SKILL.md 의 frontmatter 에 context: fork 를 추가하면 해당 스킬이 서브에이전트로 독립 실행됩니다:

```
---
name: security-audit
description: 보안 감사를 독립적으로 수행합니다.
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
model: opus
---

# 보안 감사
OWASP Top 10 기준으로 코드를 검토하세요...
```

12.5 서브에이전트 활용 패턴

🔍 **코드 리뷰 파이프라인:** 코드 작성 → code-reviewer → debugger → test-writer 순차 실행

📊 **병렬 분석:** 보안 감사, 성능 분석, 코드 품질 검사를 각각 전문 에이전트가 담당

🏗️ **레이어별 전문가:** 프론트엔드, 백엔드, DB 각 레이어의 전문 에이전트를 정의하여 분업

> 💡 **도구 제한으로 안전성 확보**
> 서브에이전트에 tools 필드로 허용 도구를 제한하면, 의도하지 않은 파일 수정이나 셸 실행을 방지할 수 있습니다.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 13

Agent Teams — 멀티 에이전트 협업 (실험적)

Agent Teams 는 2026 년 2 월 Opus 4.6 과 함께 공개된 실험적 기능입니다. 여러 Claude Code 인스턴스가 병렬로 작업하며, 공유 태스크 리스트와 직접 메시징으로 자율 협업합니다.

> ⚠ **실험적 기능**
> Agent Teams 는 기본 비활성화이며, CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 환경변수로 활성화해야 합니다. API 가 변경될 수 있고, 세션 재개·중첩 팀 등의 제한사항이 있습니다.

13.1 활성화 방법

```
# 환경변수로 활성화
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1

# 또는 settings.json 에 추가
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

13.2 아키텍처

구성 요소	역할
Team Lead	메인 세션. 팀 생성, 태스크 할당, 결과 종합
Teammates	독립 Claude 인스턴스. 각자 컨텍스트 윈도우 보유
Shared Task List	공유 태스크 목록. 팀원이 자율적으로 claim/완료
Direct Messaging	팀원 간 직접 메시징 (리드 경유 불필요)

13.3 사용법

자연어로 팀 구조를 설명하면 Claude 가 팀을 자동 생성합니다:

```
> 인증 모듈을 리팩토링하려 합니다. 에이전트 팀을 만들어주세요:
>   - API 레이어 담당 1 명
>   - 데이터베이스 마이그레이션 담당 1 명
>   - 테스트 커버리지 담당 1 명
```


© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

```
> 공유 태스크 리스트로 협업하세요.
```

13.4 효과적인 사용 사례

🔬 **리서치 & 리뷰:** 여러 팀원이 문제의 다른 측면을 동시에 조사하고, 결과를 공유·검증

🏗️ **새 모듈/기능 개발:** 각 팀원이 별도 모듈을 소유하여 파일 충돌 없이 병렬 구현

🐛 **경쟁 가설 디버깅:** 팀원마다 다른 가설을 병렬 테스트하여 빠르게 원인 수렴

🔀 **크로스 레이어 작업:** 프론트엔드, 백엔드, 테스트를 각 팀원이 담당

13.5 주요 제한사항

제한사항	설명
세션 재개 불가	/resume 으로 이전 팀원을 복원할 수 없음
태스크 상태 지연	팀원이 완료 표시를 누락할 수 있음
종료 속도	팀원이 현재 작업 완료 후 종료되어 시간 소요
팀 중첩 불가	팀원이 자체 팀을 생성할 수 없음
세션당 1 팀	새 팀 생성 전 기존 팀 정리 필요
토큰 비용	N 명 팀 = 최소 N 배 토큰 소비

13.6 베스트 프랙티스

파일 소유권 분리: 팀원마다 담당 디렉토리/파일을 명확히 분리하세요. 같은 파일 편집은 충돌을 유발합니다.

충분한 컨텍스트 제공: 팀원은 리드의 대화 기록을 상속하지 않습니다. spawn 프롬프트에 구체적인 태스크 정보를 포함하세요.

리드의 직접 구현 방지: "팀원들이 작업을 완료할 때까지 기다려"라고 명시하거나, Shift+Tab 으로 delegate 모드를 사용하세요.

팀원 수 적정화: 2~5 명이 적정합니다. 팀원당 5~6 개 태스크를 할당하세요.

읽기 작업부터 시작: 코드 리뷰 등 읽기 전용 작업으로 먼저 익숙해진 후, 구현 작업에 적용하세요.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 14

실전 워크플로우 & 베스트 프랙티스

14.1 비대화형 (Headless) 모드 자동화

CI/CD 나 스크립트에서 활용할 수 있는 비대화형 모드:

```
claude -p "보안 이슈를 검토하세요" \
  --output-format json \
  --allowedTools Read,Grep,Glob \
  --max-turns 3 \
  > security_report.json
```

14.2 멀티 디렉토리 작업

```
# 시작 시 백엔드 디렉토리 추가
claude --add-dir ../backend-api

# 세션 중 동적으로 추가
/add-dir ../shared-libraries
```

14.3 효과적인 프롬프팅 팁

🎯 **구체적으로 요청:** "JWT 인증과 비밀번호 해싱이 포함된 사용자 인증 시스템을 구현해줘"

📎 **파일 참조 활용:** @로 관련 파일을 직접 참조하면 코드베이스 스캔 시간과 토큰을 절약합니다.

🗺️ **Plan 모드 선활용:** Shift+Tab 으로 Plan 모드를 켜고 솔루션을 먼저 탐색하세요.

🧹 **/clear 습관화:** 새 작업마다 컨텍스트를 초기화하여 토큰을 절약하고 정확도를 높이세요.

📝 **CLAUDE.md 충실히 작성:** 빌드/린트 명령어, 아키텍처, 규칙을 적어두면 매번 설명할 필요가 없습니다.

14.4 팀 협업 설정

```
git add .claude/ CLAUDE.md
git commit -m "Claude Code 팀 설정 추가"
```

팀원 모두 동일한 규칙, 허용 도구, 훅, 플러그인을 공유합니다.

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 15

Claude Code 내장 도구 목록

도구	설명
Read	파일 내용 읽기
Write	파일 생성 또는 내용 쓰기
Edit	기존 파일의 특정 부분 수정
Bash	셸 명령어 실행
Grep	파일 내용에서 패턴 검색
Glob	파일명 패턴으로 파일 찾기
WebSearch	웹 검색
WebFetch	URL 에서 콘텐츠 가져오기
LSP (플러그인)	코드 정의 이동, 참조 찾기, 타입 검사

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지

CHAPTER 16

참고 자료

자료	URL
공식 문서	code.claude.com/docs
CLI 레퍼런스	code.claude.com/docs/en/cli-reference
MCP 문서	code.claude.com/docs/en/mcp
Skills 문서	code.claude.com/docs/en/skills
플러그인 가이드	code.claude.com/docs/en/discover-plugins
Agent Teams 문서	code.claude.com/docs/en/agent-teams
공식 플러그인 마켓	github.com/anthropics/claude-plugins-official
npm 패키지	www.npmjs.com/package/@anthropic-ai/claude-code
Awesome Claude Code	github.com/hesreallyhim/awesome-claude-code
Anthropic 뉴스	www.anthropic.com/news

투더제이 코딩클래스

ttj.kr

이 가이드북은 투더제이 코딩클래스에서 제작하여 배포하였습니다.

본 자료는 자유롭게 복제 및 재배포할 수 있습니다.

단, 무단으로 수정하여 배포하거나 상업적 목적으로 사용하는 것은 금지됩니다.

2026 년2 월 기준 · Anthropic 공식 문서 기반 · 최신 정보는 code.claude.com/docs 참고

© 투더제이 코딩클래스 ttj.kr · 복제 및 재배포 가능 | 무단 수정 배포 및 상업적 사용 금지